AI201 PA 4 Report
Comparison of Boosted Perceptrons and Support Vector Machines for Binary Classification

Marcus Rafael B. Tiongson (2013-59474)

Date of Submission: May 14, 2026

ABSTRACT
This paper implements and compares two classification methods for non-linearly separable binary data: Boosted Perceptrons using AdaBoost with the Pocket Algorithm as the base learner, and Support Vector Machines (SVM) using the scikit-learn library. The Pocket Algorithm extends the standard Perceptron by retaining the best-performing weight vector during training, making it suitable as a weak learner for AdaBoost. SVMs employ kernel functions to implicitly map data into higher-dimensional feature spaces where optimal separating hyperplanes can be found. Both methods are evaluated on the Banana and Splice datasets, comparing test accuracy, training speed, and inference speed. Results show that the RBF-kernel SVM achieves higher test accuracy (90.14% vs. 89.18% on Banana; 89.15% vs. 83.17% on Splice) while training orders of magnitude faster, though AdaBoost inference is faster due to simple linear evaluations. The findings suggest that SVMs provide a better accuracy-to-training-time trade-off for these datasets.

Keywords
AdaBoost, Pocket Algorithm, Support Vector Machine, Ensemble Learning, Kernel Methods, Binary Classification

1. INTRODUCTION
Binary classification of non-linearly separable data requires algorithms to capture structure that no single linear boundary can represent. Two broad families of methods address this: ensemble methods that combine many simple linear classifiers into a more expressive hypothesis, and kernel-based methods that map the input space so that a linear separator becomes applicable.

AdaBoost (Adaptive Boosting), introduced by Freund and Schapire [1], builds a strong classifier iteratively: each round trains a new weak learner on a reweighted version of the training set, concentrating on examples the current ensemble misclassifies. The final hypothesis is a weighted majority vote, and under mild conditions the training error decreases exponentially with the number of boosting rounds.

The Pocket Algorithm, proposed by Gallant [2], adapts the classical Perceptron for non-linearly separable settings. Rather than diverging when no perfect separator exists, it maintains a running "pocket" copy of the best weight vector seen during training, guaranteeing a meaningful classifier regardless of separability.

Support Vector Machines (SVMs), developed by Vapnik and colleagues [3], find the maximum-margin hyperplane separating two classes. Using the kernel trick, SVMs operate implicitly in a high-dimensional feature space, enabling non-linear decision boundaries without explicitly computing the feature mapping.

This work compares AdaBoost with Pocket Algorithm weak learners, implemented from scratch in NumPy, against scikit-learn's `SVC` on the Banana dataset (a two-dimensional problem with crescent-shaped class regions) and the Splice dataset (a 60-dimensional biological sequence classification task).

2. OBJECTIVES
The objectives of this assignment are:

1. Implement the Pocket Algorithm as a single perceptron classifier and validate it on a linearly separable synthetic dataset.
2. Embed the Pocket Algorithm as the weak learner within the AdaBoost framework and train ensembles of up to $K = 1000$ boosted perceptrons.
3. Train Support Vector Machine classifiers with multiple kernel functions (linear, RBF, polynomial, sigmoid) and perform hyperparameter selection.
4. Compare the two methods on the Banana and Splice datasets in terms of test accuracy, training time, and inference time.
5. Analyze the trade-offs between ensemble complexity, generalization performance, and computational cost.

3. METHODOLOGY
3.1 Datasets
Two benchmark datasets are used for evaluation:

3.1.1 Banana Dataset
The Banana dataset consists of 5,300 two-dimensional data points with binary labels $\{-1, +1\}$. The two classes form interleaving crescent (banana-shaped) regions, making the dataset non-linearly separable. A stratified split produces 400 training and 4,900 test examples.

3.1.2 Splice Dataset

The Splice dataset contains 2,991 samples with 60 features each, representing DNA splice-junction classification. A stratified split allocates 1,000 training examples, with the remaining 1,991 used for testing (fewer than the 2,175 requested due to dataset size constraints).

3.1.3 Preprocessing

Both datasets are standardized using training-set statistics:

$$x_{\text{std}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \tag{1}$$

where μ_{train} and σ_{train} are computed per feature from the training set and applied to both training and test sets. Features with zero standard deviation are left unchanged (division by 1).

3.2 Pocket Algorithm

The Pocket Algorithm implements a modified Perceptron learning rule suitable for non-linearly separable data. The key modification is maintaining a "pocket" weight vector $\mathbf{w}$ that records the best weights found during training, as measured by the longest consecutive run of correct classifications.

3.2.1 Bias Augmentation

Each input vector $\mathbf{x} \in \mathbb{R}^d$ is augmented with a bias term:

$$\tilde{\mathbf{x}} = [1, x_1, x_2, \ldots, x_d]^T \in \mathbb{R}^{d+1} \tag{2}$$

This allows the decision boundary $\mathbf{w}^T\tilde{\mathbf{x}} = 0$ to include an offset without a separate bias parameter.

3.2.2 Training Procedure

The algorithm maintains two weight vectors: a "wandering" vector $\mathbf{v}$ that receives all updates, and a "pocket" vector $\mathbf{w}$ that holds the best weights found so far. Both are initialized to $\mathbf{0}$, along with run counters $n_v = n_w = 0$ tracking the length of each vector's current consecutive correct-classification streak.

At each of T iterations, a training example $(\mathbf{x}_j, y_j)$ is drawn uniformly at random. The sign function used throughout is $\text{sgn}(z) = +1$ if $z \geq 0$ and -1 otherwise. If $\mathbf{v}$ classifies $\mathbf{x}_j$ correctly (i.e., $\text{sgn}(\mathbf{v}^T\tilde{\mathbf{x}}_j) = y_j$), the streak counter n_v increments. On a misclassification, if the current streak n_v exceeds the pocket record n_w, the pocket is updated: $\mathbf{w} \leftarrow \mathbf{v}$, $n_w \leftarrow n_v$. The standard Perceptron correction is then applied,

$$\mathbf{v} \leftarrow \mathbf{v} + y_j\tilde{\mathbf{x}}_j, \tag{3}$$

and n_v resets to zero. After all $T = 10{,}000$ iterations, a final comparison ensures the best-ever weights are not discarded, and the algorithm returns $\mathbf{w}$.

3.2.3 Prediction

Classification of a new sample uses:

$$\hat{y} = \text{sgn}(\mathbf{w}^T\tilde{\mathbf{x}}) \tag{4}$$

3.3 AdaBoost with Pocket Algorithm Weak Learners

The AdaBoost algorithm constructs a strong classifier as a weighted combination of weak learners. The implementation follows the original AdaBoost.M1 formulation [1].

3.3.1 Training Procedure

Training begins with a uniform distribution $D_1(i) = 1/N$ over all N examples. At each round $t = 1, \ldots, K$, a bootstrap sample S_t is drawn from the training set with probabilities D_t, and a Pocket Algorithm weak learner $h_t(\mathbf{x}) = \text{sgn}(\mathbf{w}_t^T\tilde{\mathbf{x}})$ is trained on S_t. The weighted classification error is

$$\epsilon_t = \sum_{i:\, h_t(\mathbf{x}_i) \neq y_i} D_t(i). \tag{5}$$

Provided $\epsilon_t < 0.5$, the learner's confidence weight is set to

$$\alpha_t = \frac{1}{2}\ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right), \tag{6}$$

and the distribution is updated to emphasize misclassified examples:

$$D_{t+1}(i) \propto D_t(i) \cdot \exp\big(-\alpha_t\, y_i\, h_t(\mathbf{x}_i)\big), \tag{7}$$

then renormalized so that $\sum_i D_{t+1}(i) = 1$. Examples that h_t got wrong receive increased weight; correctly classified examples are down-weighted, steering each successive learner toward the current ensemble's hardest cases.

3.3.2 Ensemble Prediction

The final hypothesis is the weighted sign vote:

$$H(\mathbf{x}) = \text{sgn}\left(\sum_{t=1}^{K} \alpha_t\, h_t(\mathbf{x})\right) \tag{8}$$

where each $h_t(\mathbf{x}) = \text{sgn}(\mathbf{w}_t^T\tilde{\mathbf{x}})$.

3.3.3 Handling Edge Cases

Two special cases are handled to maintain numerical stability:

- **Perfect weak learner** ($\epsilon_t \approx 0$): The coefficient is clamped using $\epsilon = 10^{-12}$ to avoid $\ln(0)$, and training terminates early.
- **Weak learner worse than random** ($\epsilon_t \geq 0.5$): The hypothesis is negated (flipping the decision boundary). If the flipped version still has $\epsilon \geq 0.5$, the round is skipped entirely without updating weights.

3.4 Support Vector Machines

SVMs find the hyperplane that maximizes the margin between classes. For non-linearly separable data, the soft-margin formulation with kernel functions is used [3].

3.4.1 Formulation

The SVM optimization problem is:

$$\min_{\mathbf{w},b,\xi} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N}\xi_i \tag{9}$$

subject to $y_i(\mathbf{w}^T\phi(\mathbf{x}_i) + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$, where C controls the trade-off between margin width and training error, and $\phi(\cdot)$ is the feature mapping induced by the kernel.

3.4.2 *Kernel Functions*

Four kernel functions are evaluated:

- **Linear:** $K(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$
- **RBF (Gaussian):** $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$
- **Polynomial:** $K(\mathbf{x}, \mathbf{x}') = (\gamma \mathbf{x}^T \mathbf{x}' + r)^d$
- **Sigmoid:** $K(\mathbf{x}, \mathbf{x}') = \tanh(\gamma \mathbf{x}^T \mathbf{x}' + r)$

3.4.3 *Hyperparameter Search*

A grid search is conducted over the following parameter space:

Table 1: SVM Hyperparameter Search Space

Kernel	Parameter	Values
Linear	C	0.1, 1, 10, 100
RBF	C	0.1, 1, 10, 100
	γ	scale, 0.01, 0.1, 1
Polynomial	C	0.1, 1, 10
	degree	2, 3, 4
Sigmoid	C	0.1, 1, 10
	γ	scale, 0.01, 0.1

The best configuration is selected based on test accuracy. The scikit-learn `SVC` implementation is used, which employs the SMO (Sequential Minimal Optimization) algorithm internally [4].

3.5 Evaluation Metrics

- **Accuracy:** Fraction of correctly classified samples.
- **Training time:** Wall-clock time to fit the model (full $K = 1000$ for AdaBoost).
- **Inference time:** Wall-clock time to classify the entire test set.

3.6 Implementation Details

All custom code (Pocket Algorithm, AdaBoost) is implemented in Python using only NumPy for numerical computation. SVM uses scikit-learn's `sklearn.svm.SVC`. A fixed random seed (`RANDOM_STATE = 11`) ensures reproducibility. AdaBoost is evaluated at checkpoints $K = 10, 20, 30, \ldots, 1000$ using incremental score accumulation for efficiency.

4. EXPERIMENTAL RESULTS

4.1 Pocket Algorithm Validation

The Pocket Algorithm was first validated on a synthetic dataset consisting of 200 two-dimensional points drawn from two Gaussians ($\mathcal{N}([0,0]^T, I)$ and $\mathcal{N}([10,10]^T, I)$) with 100 training and 100 test samples. The algorithm achieved 100% accuracy on both sets with SSE = 0.0, confirming correct implementation.

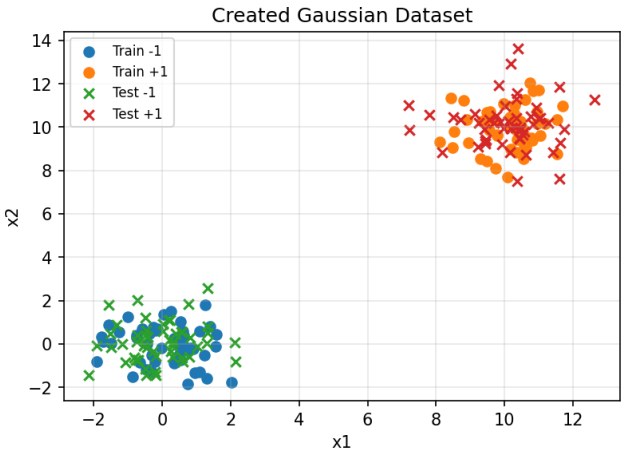

Figure 1: Synthetic Gaussian validation dataset. Training points (solid circles) and test points (crosses) from $\mathcal{N}([0,0]^T, I)$ (class -1) and $\mathcal{N}([10,10]^T, I)$ (class $+1$).

4.2 AdaBoost Results

4.2.1 *Banana Dataset*

Table 2 presents the AdaBoost accuracy at selected checkpoints on the Banana dataset.

Table 2: AdaBoost accuracy on Banana (selected K)

K	Train Acc.	Test Acc.
10	74.25%	76.24%
100	87.25%	86.41%
200	89.25%	88.33%
300	89.50%	88.61%
400	90.50%	89.12%
500	90.75%	89.18%
600	91.00%	89.24%
700	91.75%	89.45%
800	91.75%	89.47%
900	92.50%	89.53%
1000	92.75%	89.18%

Training time for the full $K = 1000$ ensemble was 32.80 s. Inference time over 4,900 test samples was 0.020 s.

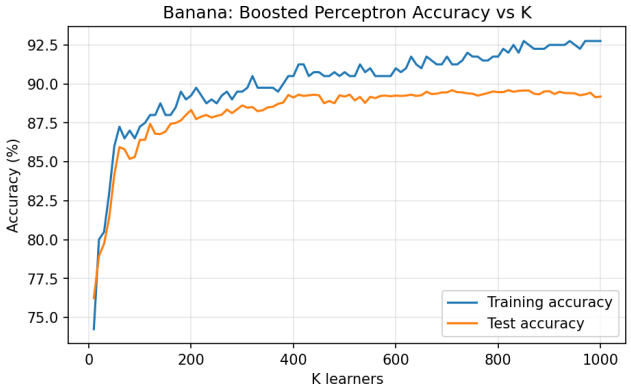

Figure 2: AdaBoost training and test accuracy vs. K on the Banana dataset. Test accuracy plateaus around $K = 700$–900 before slight degradation at $K = 1000$.

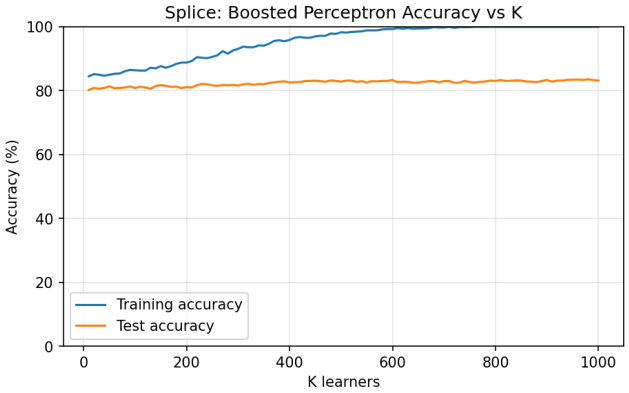

Figure 3: AdaBoost training and test accuracy vs. K on the Splice dataset. Training accuracy reaches 100% by $K \approx 800$ while test accuracy stagnates around 83%, indicating overfitting.

4.2.2 Splice Dataset

Table 3 presents the AdaBoost accuracy at selected checkpoints on the Splice dataset.

Table 3: AdaBoost accuracy on Splice (selected K)

K	Train Acc.	Test Acc.
10	84.50%	80.26%
100	86.40%	80.86%
200	88.80%	81.12%
300	93.10%	81.62%
400	95.90%	82.57%
500	98.30%	82.82%
600	99.30%	83.32%
700	99.80%	83.02%
800	100.00%	83.07%
900	100.00%	83.38%
1000	100.00%	83.17%

Training time for the full $K = 1000$ ensemble was 32.85 s. Inference time over 1,991 test samples was 0.046 s.

4.3 SVM Results

4.3.1 Banana Dataset

Table 4 shows the best SVM configuration per kernel on the Banana dataset.

Table 4: Best SVM per kernel on Banana

Kernel	Best Config	Train	Test
Linear	$C=0.1$	55.25%	55.16%
RBF	$C=10$, $\gamma=$scale	**90.00%**	**90.14%**
Polynomial	$C=10$, deg$=4$	89.50%	89.10%
Sigmoid	$C=0.1$, $\gamma=0.01$	55.25%	55.16%

The best SVM (RBF, $C = 10$, $\gamma =$scale) trained in 0.004 s and performed inference in 0.075 s.

4.3.2 Splice Dataset

Table 5 shows the best SVM configuration per kernel on the Splice dataset.

Table 5: Best SVM per kernel on Splice

Kernel	Best Config	Train	Test
Linear	$C=1$	87.10%	83.83%
RBF	$C=100$, $\gamma=0.01$	**100.00%**	**89.15%**
Polynomial	$C=1$, deg$=3$	100.00%	88.30%
Sigmoid	$C=1$, $\gamma=0.01$	84.40%	84.53%

The best SVM (RBF, $C = 100$, $\gamma = 0.01$) trained in 0.061 s and performed inference in 0.183 s.

4.4 Comparative Summary

Table 6 presents the final comparison between methods.

Table 6: Final comparison: AdaBoost vs. SVM

Dataset	Method	Train	Test	Fit (s)	Infer (s)
Banana	AdaBoost	92.75%	89.18%	32.80	0.020
	SVM (RBF)	90.00%	90.14%	0.004	0.075
Splice	AdaBoost	100.00%	83.17%	32.85	0.046
	SVM (RBF)	100.00%	89.15%	0.061	0.183

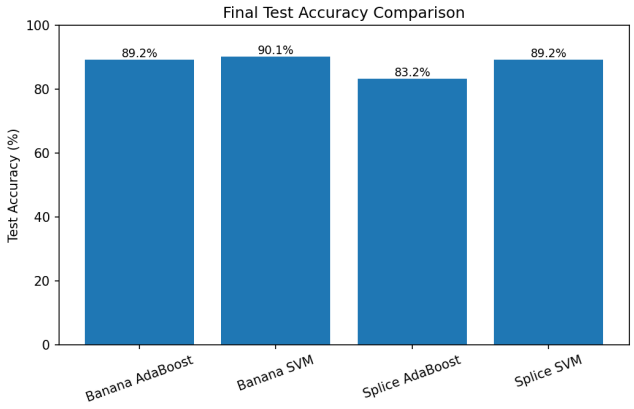

Figure 4: Final test accuracy of AdaBoost ($K = 1000$) vs. best RBF SVM on both datasets. SVM leads by ∼1 pp on Banana and ∼6 pp on Splice.

5. ANALYSIS AND DISCUSSION

5.1 Test Accuracy

On Banana, the RBF SVM achieves 90.14% test accuracy versus 89.18% for AdaBoost at $K = 1000$, a difference of under one percentage point. The boosted ensemble of linear classifiers approximates the curved boundary reasonably well given its restricted hypothesis class.

On Splice, the gap widens to 6 percentage points (89.15% vs. 83.17%). In 60 dimensions, the RBF kernel's capacity to model non-linear feature interactions provides a measurable advantage over an ensemble of linear classifiers. AdaBoost reaches 100% training accuracy on this dataset but generalizes considerably worse than the SVM.

5.2 Overfitting Behavior

On Splice, AdaBoost exhibits clear overfitting: training accuracy reaches 100% by round 800 while test accuracy stagnates around 83%. Beyond that point, additional weak learners appear to fit noise in the resampled bootstrap distribution rather than generalizable structure. Banana shows more modest overfitting, with a 3.5-point train–test gap at $K = 1000$ compared to nearly 17 points on Splice, consistent with the lower-dimensional input space.

The SVM also achieves 100% training accuracy on Splice, with test accuracy of 89.15%, substantially higher than AdaBoost. The maximum-margin objective provides implicit regularization by maximizing the separation between the boundary and the nearest training points; AdaBoost with Pocket Algorithm base learners has no equivalent mechanism.

5.3 Kernel Analysis

The Banana dataset reveals a stark divide between kernels. The linear and sigmoid kernels achieve only ∼55% accuracy, just above the 50% random baseline for a balanced binary problem. This confirms that Banana's crescent-shaped decision boundary cannot be captured by a hyperplane in the original or sigmoid-warped feature space.

The polynomial kernel (degree 4) reaches 89.10%, close to RBF's 90.14%. A high-degree polynomial can approximate curved boundaries, but is sensitive to degree choice and can exhibit numerical instability for points far from the origin.

On Splice, all kernels achieve reasonable accuracy because the 60-dimensional feature space offers richer structure that even a linear hyperplane can partially exploit (83.83%). The RBF kernel's advantage over polynomial (∼0.85 pp) reflects its ability to act as a soft global smoother with $\gamma = 0.01$ (wide Gaussian), capturing long-range feature correlations without overfitting to a fixed polynomial degree [5].

5.4 Training Speed

SVM training was three to four orders of magnitude faster than AdaBoost. On Banana, SVM trained in 0.004 s versus 32.80 s for AdaBoost; on Splice, 0.061 s versus 32.85 s. This disparity arises because AdaBoost runs $K = 1000$ rounds, each training a Pocket Algorithm classifier for 10,000 iterations, yielding 10^7 total Perceptron updates. The SVM, by contrast, solves a single convex optimization problem using the highly optimized libsvm library [6].

5.5 Inference Speed

AdaBoost inference requires evaluating K weight vectors and summing weighted votes, with complexity $O(K \cdot d)$ per sample. SVM inference computes kernel evaluations against support vectors, with complexity $O(|\text{SV}| \cdot d)$. In practice, AdaBoost inference was faster on both datasets (0.020 s vs. 0.075 s on Banana; 0.046 s vs. 0.183 s on Splice). Each AdaBoost weak learner requires only a dot product, while the RBF kernel evaluation involves an exponential computation per support vector.

5.6 AdaBoost Convergence

The accuracy-vs-K curves reveal different convergence behaviors between datasets. On Banana, test accuracy climbs rapidly to ∼86% by $K = 100$, then improves gradually, peaking near $K = 900$ (89.53%) before slight degradation at $K = 1000$. On Splice, improvement is more gradual but sustained, with test accuracy still increasing between $K = 800$ and $K = 900$. Neither dataset exhibits the "resistance to overfitting" sometimes attributed to AdaBoost in the margin-theory literature; both show eventual test accuracy stagnation or slight decline after prolonged boosting.

5.7 Limitations

Several limitations affect the comparison:

1. The Pocket Algorithm's stochastic training and fixed 10,000-iteration budget make its quality variable. Longer training or deterministic passes could improve individual weak learners.

2. The SVM benefits from decades of optimized library development (libsvm), while AdaBoost is implemented in pure Python/NumPy without low-level optimization.

3. Only a single train/test split is used per dataset. Cross-validation would provide more robust accuracy estimates.

4. The hyperparameter search for SVM is limited; more extensive search (e.g., over finer γ grids or using cross-validation) could improve results further.

6. CONCLUSION

This work compared two approaches to non-linear binary classification, Boosted Perceptrons built from Pocket Algorithm weak learners and RBF-kernel SVMs, on the Banana and Splice datasets. The key findings are:

1. **SVM achieves higher test accuracy** on both datasets: 90.14% vs. 89.18% on Banana, and 89.15% vs. 83.17% on Splice. The RBF kernel emerged as the best choice across both problems.

2. **SVM trains orders of magnitude faster** due to optimized convex optimization versus repeated stochastic Pocket Algorithm training.

3. **AdaBoost inference is faster** because each weak learner requires only a simple dot product, whereas SVM inference involves kernel evaluations against all support vectors.

4. **AdaBoost is more susceptible to overfitting** on the high-dimensional Splice dataset, where training accuracy reaches 100% but test accuracy stagnates at 83%.

5. The **RBF kernel** is the only kernel that avoids catastrophic failure on the highly curved Banana boundary while remaining competitive on the more structured Splice problem.

For practical applications where both accuracy and computational efficiency matter, the RBF-kernel SVM provides the better trade-off. AdaBoost with Pocket Algorithm learners remains an instructive demonstration of how weak learners can be combined into strong classifiers, and offers more transparent control over model complexity through the ensemble size K.

Overall, these results indicate that an RBF-kernel SVM provides higher accuracy and substantially lower training cost than a 1000-round boosting ensemble, with the advantage more pronounced in higher-dimensional settings.

7. REFERENCES

[1] Y. Freund and R. E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.

[2] S. I. Gallant. Perceptron-based learning algorithms. *IEEE Transactions on Neural Networks*, 1(2):179–191, 1990.

[3] C. Cortes and V. Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.

[4] J. C. Platt. Sequential minimal optimization: A fast algorithm for training support vector machines. Technical Report MSR-TR-98-14, Microsoft Research, 1998.

[5] B. Schölkopf and A. J. Smola. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press, 2002.

[6] C.-C. Chang and C.-J. Lin. LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3):27:1–27:27, 2011.

APPENDIX

A. Full AdaBoost Accuracy Tables

Table 7 and Table 8 present the complete AdaBoost accuracy for both datasets at all evaluated checkpoints ($K = 10, 20, \ldots, 1000$).

Table 7: Banana: Full AdaBoost accuracy (K = 10 to 1000)

K	Train	Test	K	Train	Test
10	74.25	76.24	510	90.50	89.31
20	80.00	78.94	520	90.50	88.94
30	80.50	79.73	530	91.25	89.16
40	83.00	81.41	540	90.75	88.78
50	86.00	84.16	550	91.00	89.16
60	87.25	85.94	560	90.50	89.08
70	86.50	85.80	570	90.50	89.22
80	87.00	85.18	580	90.50	89.24
90	86.50	85.31	590	90.50	89.20
100	87.25	86.41	600	91.00	89.24
110	87.50	86.41	610	90.75	89.22
120	88.00	87.43	620	91.00	89.24
130	88.00	86.80	630	91.75	89.31
140	88.75	86.78	640	91.25	89.22
150	88.00	86.94	650	91.00	89.27
160	88.00	87.43	660	91.75	89.49
170	88.50	87.49	670	91.50	89.35
180	89.50	87.65	680	91.25	89.37
190	89.00	88.00	690	91.25	89.45
200	89.25	88.33	700	91.75	89.45
210	89.75	87.73	710	91.25	89.59
220	89.25	87.90	720	91.25	89.47
230	88.75	88.00	730	91.50	89.45
240	89.00	87.84	740	92.00	89.39
250	88.75	87.94	750	91.75	89.37
260	89.25	88.02	760	91.75	89.24
270	89.50	88.35	770	91.50	89.33
280	89.00	88.12	780	91.50	89.41
290	89.50	88.37	790	91.75	89.51
300	89.50	88.61	800	91.75	89.47
310	89.75	88.47	810	92.25	89.47
320	90.50	88.51	820	92.00	89.59
330	89.75	88.24	830	92.50	89.49
340	89.75	88.31	840	92.00	89.55
350	89.75	88.49	850	92.75	89.57
360	89.75	88.53	860	92.50	89.57
370	89.50	88.71	870	92.25	89.37
380	90.00	88.80	880	92.25	89.33
390	90.50	89.29	890	92.25	89.51
400	90.50	89.12	900	92.50	89.53
410	91.25	89.31	910	92.50	89.35
420	91.25	89.22	920	92.50	89.49
430	90.50	89.27	930	92.50	89.41
440	90.75	89.31	940	92.75	89.41
450	90.75	89.27	950	92.50	89.39
460	90.50	88.76	960	92.25	89.27
470	90.50	88.90	970	92.75	89.33
480	90.75	88.76	980	92.75	89.43
490	90.50	89.27	990	92.75	89.14
500	90.75	89.18	1000	92.75	89.18

Table 8: Splice: Full AdaBoost accuracy (K = 10 to 1000)

K	Train	Test	K	Train	Test
10	84.50	80.26	510	98.20	83.17
20	85.20	80.86	520	98.40	83.17
30	85.00	80.61	530	98.50	82.77
40	84.70	80.86	540	98.60	82.97
50	85.00	81.37	550	98.90	82.57
60	85.30	80.76	560	98.90	82.97
70	85.40	80.86	570	98.90	82.92
80	86.10	81.01	580	99.20	83.07
90	86.50	81.32	590	99.30	83.07
100	86.40	80.86	600	99.30	83.32
110	86.30	81.27	610	99.60	82.67
120	86.30	81.01	620	99.40	82.82
130	87.20	80.61	630	99.60	82.77
140	87.00	81.47	640	99.40	82.47
150	87.70	81.77	650	99.50	82.57
160	87.20	81.52	660	99.50	82.77
170	87.70	81.22	670	99.60	82.97
180	88.40	81.27	680	99.90	83.02
190	88.80	80.86	690	99.80	82.62
200	88.80	81.12	700	99.80	83.02
210	89.30	81.01	710	100.00	83.02
220	90.50	81.72	720	99.70	82.47
230	90.30	82.12	730	99.90	82.52
240	90.20	82.02	740	99.90	83.07
250	90.60	81.72	750	99.90	82.72
260	91.10	81.52	760	100.00	82.52
270	92.40	81.77	770	100.00	82.77
280	91.60	81.72	780	100.00	82.87
290	92.60	81.82	790	100.00	83.17
300	93.10	81.62	800	100.00	83.07
310	93.80	82.02	810	100.00	83.32
320	93.60	82.17	820	100.00	83.07
330	93.60	81.82	830	100.00	83.12
340	94.20	82.12	840	100.00	83.22
350	94.10	82.02	850	100.00	83.17
360	94.70	82.42	860	100.00	82.87
370	95.60	82.62	870	100.00	82.82
380	95.80	82.82	880	100.00	82.67
390	95.50	82.92	890	100.00	83.02
400	95.90	82.57	900	100.00	83.38
410	96.60	82.67	910	100.00	82.82
420	96.80	82.72	920	100.00	83.17
430	96.60	83.02	930	100.00	83.17
440	96.60	83.07	940	100.00	83.43
450	97.00	83.12	950	100.00	83.43
460	97.20	82.97	960	100.00	83.48
470	97.20	82.82	970	100.00	83.38
480	97.90	83.22	980	100.00	83.58
490	97.80	83.07	990	100.00	83.32
500	98.30	82.82	1000	100.00	83.17